

❖ ДОМЕН 7: DEVOPS, КОНТЕЙНЕРЫ И ИНФРАСТРУКТУРА — ПОЛНАЯ ДЕТАЛИЗАЦИЯ (ОТ НУЛЯ ДО ЭКСПЕРТА)

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 7.1: ОСНОВЫ LINUX И BASH (JUNIOR LEVEL)

7.1.1. Linux (Junior Level)

№ 7.1.1.1 — Файловая система

- **Что нужно знать:** структура (/, /etc, /var, /usr, /home, /tmp, /dev, /proc)
- **Вопросы на собеседовании:** "Какая структура каталогов в Linux?"
- **Практика:** Нарисовать дерево файловой системы и объяснить назначение каждой папки

№ 7.1.1.2 — Навигация

- **Что нужно знать:** pwd, ls, cd, mkdir, rmdir, tree
- **Вопросы на собеседовании:** "Как перейти в домашнюю директорию?"
- **Практика:** Создать структуру project/src, project/build, project/docs и заполнить файлами

№ 7.1.1.3 — Работа с файлами

- **Что нужно знать:** touch, cp, mv, rm, ln (жесткие и символьные ссылки)
- **Вопросы на собеседовании:** "В чем разница между жесткой и символической ссылкой?"
- **Практика:** Создать жесткую ссылку и символическую ссылку, показать разницу

№ 7.1.1.4 — Права доступа

- **Что нужно знать:** chmod (числовой и символический режим), chown, chgrp, umask
- **Вопросы на собеседовании:** "Что означают права rwxr-xr--?"

- **Практика:** Создать файл, изменить права на 755, объяснить каждый бит

№ 7.1.1.5 — Просмотр файлов

- **Что нужно знать:** cat, less/more, head, tail
- **Вопросы на собеседовании:** "Как посмотреть последние 20 строк лог-файла?"
- **Практика:** Открыть лог-файл, использовать tail -f для просмотра в реальном времени

№ 7.1.1.6 — Процессы

- **Что нужно знать:** ps, top/htop, kill, kill -9, nice
- **Вопросы на собеседовании:** "Как завершить процесс по PID?"
- **Практика:** Запустить бесконечный цикл в терминале, найти его PID через ps aux, завершить через kill

№ 7.1.1.7 — Переменные окружения

- **Что нужно знать:** export, env, echo \$PATH, .bashrc, .bash_profile
- **Вопросы на собеседовании:** "Как добавить директорию в PATH?"
- **Практика:** Добавить /usr/local/bin в PATH в .bashrc

7.1.2. Bash (Junior → Middle Level)

№ 7.1.2.1 — Переменные

- **Что нужно знать:** объявление (VAR="value"), использование (\$VAR), кавычки (' vs ")
- **Вопросы на собеседовании:** "В чем разница между '\$VAR' и "\$VAR"?"
- **Практика:** Написать скрипт, выводящий приветствие с именем пользователя

№ 7.1.2.2 — Условные операторы

- **Что нужно знать:** if, elif, else, case, test ([]), [[]]
- **Вопросы на собеседовании:** "Как проверить существование файла?"
- **Практика:** Написать скрипт, проверяющий наличие config.ini и создающий его при отсутствии

№ 7.1.2.3 — Циклы

- **Что нужно знать:** for, while, until
- **Вопросы на собеседовании:** "Как перебрать все файлы в папке?"
- **Практика:** Написать скрипт, переименовывающий все .txt в .bak

№ 7.1.2.4 — Функции

- **Что нужно знать:** объявление, параметры (\$1, \$2, \$@), возврат (return, echo)
- **Вопросы на собеседовании:** "Как передать аргументы в Bash-функцию?"
- **Практика:** Написать функцию log_error, записывающую сообщение в лог

№ 7.1.2.5 — Ввод-вывод

- **Что нужно знать:** перенаправление (> vs >>), конвейер (|), read
- **Вопросы на собеседовании:** "Что делает grep error log.txt | wc -l?"
- **Практика:** Написать скрипт, читающий ввод пользователя и обрабатывающий его

№ 7.1.2.6 — grep

- **Что нужно знать:** поиск по регулярным выражениям, -i (регистронезависимый), -r (рекурсивный)
- **Вопросы на собеседовании:** "Как найти все файлы, содержащие 'error'?"
- **Практика:** Написать скрипт, ищущий error во всех .log файлах

№ 7.1.2.7 — awk

- **Что нужно знать:** обработка колонок, awk '{print \$2}'
- **Вопросы на собеседовании:** "Как вывести второй столбец из CSV?"
- **Практика:** Написать скрипт, извлекающий второй столбец из CSV

№ 7.1.2.8 — sed

- **Что нужно знать:** замена текста, sed 's/old/new/g'
 - **Вопросы на собеседовании:** "Как заменить все вхождения 'foo' на 'bar'?"
 - **Практика:** Написать скрипт, заменяющий текст в файле
-

7.1.3. Практические задания (Junior → Middle Level)

№ 7.1.3.1

- **Задание:** Написать скрипт для бэкапа логов (архивация и удаление старых)
- **Что проверяет:** Знание Bash и работы с файлами

№ 7.1.3.2

- **Задание:** Написать скрипт, который находит все файлы .log старше 30 дней и архивирует их
- **Что проверяет:** Знание find, tar, gzip

№ 7.1.3.3

- **Задание:** Написать скрипт, который парсит логи и выводит топ-10 IP-адресов по количеству запросов
- **Что проверяет:** Знание awk, sort, uniq

РАЗДЕЛ 7.2: DOCKER (JUNIOR → SENIOR)

7.2.1. Docker (Junior Level)

№ 7.2.1.1 — Что такое Docker

- **Что нужно знать:** контейнеризация vs виртуализация, образы, контейнеры, реестры
- **Вопросы на собеседовании:** "В чем разница между контейнером и виртуальной машиной?"
- **Практика:** Объяснить на схеме разницу между VM и контейнером

№ 7.2.1.2 — Dockerfile

- **Что нужно знать:** инструкции FROM, RUN, COPY, ADD, CMD, ENTRYPOINT, EXPOSE, WORKDIR
- **Вопросы на собеседовании:** "Какие инструкции есть в Dockerfile?"
- **Практика:** Написать Dockerfile для C++ "Hello World"

№ 7.2.1.3 — docker build

- **Что нужно знать:** сборка образа, -t (тег), кэширование слоев
- **Вопросы на собеседовании:** "Как собрать Docker-образ?"

- **Практика:** Собрать образ с тегом myapp:latest

№ 7.2.1.4 — docker run

- **Что нужно знать:** запуск контейнера, -d (detached), -it (interactive), --rm
- **Вопросы на собеседовании:** "Как запустить контейнер в фоновом режиме?"
- **Практика:** Запустить контейнер в фоне и посмотреть логи

№ 7.2.1.5 — docker ps

- **Что нужно знать:** список запущенных контейнеров, -a (все), -q (только ID)
- **Вопросы на собеседовании:** "Как посмотреть все запущенные контейнеры?"
- **Практика:** Запустить несколько контейнеров и показать их список

№ 7.2.1.6 — docker stop/start/restart

- **Что нужно знать:** управление контейнерами
- **Вопросы на собеседовании:** "Как остановить контейнер?"
- **Практика:** Остановить и запустить контейнер

№ 7.2.1.7 — docker exec

- **Что нужно знать:** выполнение команд внутри контейнера
- **Вопросы на собеседовании:** "Как зайти внутрь контейнера?"
- **Практика:** Зайти в контейнер с docker exec -it container bash

№ 7.2.1.8 — docker logs

- **Что нужно знать:** просмотр логов, -f (follow)
- **Вопросы на собеседовании:** "Как посмотреть логи контейнера?"
- **Практика:** Просмотреть логи и использовать -f

№ 7.2.1.9 — Volumes

- **Что нужно знать:** -v (монтирование папок), docker volume create
- **Вопросы на собеседовании:** "Как сохранить данные после удаления контейнера?"
- **Практика:** Создать том и примонтировать его к контейнеру

№ 7.2.1.10 — Сети

- **Что нужно знать:** bridge (по умолчанию), host, overlay, docker network create
 - **Вопросы на собеседовании:** "Как связать два контейнера?"
 - **Практика:** Создать сеть и запустить два контейнера в ней
-

7.2.2. Docker (Middle Level)

№ 7.2.2.1 — Multi-stage сборка

- **Что нужно знать:** разделение build и runtime слоев
- **Вопросы на собеседовании:** "Как уменьшить размер Docker-образа?"
- **Практика:** Написать Dockerfile с multi-stage для C++ приложения

№ 7.2.2.2 — Healthcheck

- **Что нужно знать:** HEALTHCHECK для проверки готовности приложения
- **Вопросы на собеседовании:** "Как проверить, что приложение в контейнере работает?"
- **Практика:** Добавить healthcheck для API

№ 7.2.2.3 — .dockerignore

- **Что нужно знать:** исключение файлов из контекста сборки
- **Вопросы на собеседовании:** "Какие файлы стоит исключить из Docker-образа?"
- **Практика:** Создать .dockerignore для C++ проекта (игнорировать build/, .git)

№ 7.2.2.4 — Оптимизация кэширования

- **Что нужно знать:** порядок инструкций для максимального кэширования
- **Вопросы на собеседовании:** "Как ускорить сборку Docker-образа?"
- **Практика:** Переписать Dockerfile, чтобы кэшировались зависимости

№ 7.2.2.5 — Non-root user

- **Что нужно знать:** безопасный запуск контейнера
- **Вопросы на собеседовании:** "Почему не стоит запускать контейнер от root?"

- **Практика:** Создать пользователя и запустить приложение от него

№ 7.2.2.6 — Docker Compose

- **Что нужно знать:** определение многоконтейнерного приложения в docker-compose.yml
- **Вопросы на собеседовании:** "Как запустить несколько контейнеров вместе?"
- **Практика:** Создать compose с приложением + PostgreSQL + Redis

№ 7.2.2.7 — Docker Compose (depends_on)

- **Что нужно знать:** порядок запуска контейнеров
- **Вопросы на собеседовании:** "Как гарантировать, что БД запустится до приложения?"
- **Практика:** Использовать depends_on в compose

№ 7.2.2.8 — Docker Compose (environment)

- **Что нужно знать:** передача переменных окружения
- **Вопросы на собеседовании:** "Как передать конфигурацию в контейнер?"
- **Практика:** Передать строку подключения к БД через переменные

7.2.3. Docker (Senior Level)

№ 7.2.3.1 — Cgroups и Namespaces

- **Что нужно знать:** как Docker изолирует контейнеры (pid, net, ipc, mnt, uts, user)
- **Вопросы на собеседовании:** "Как работает изоляция в Docker?"
- **Практика:** Изучить lsns и показать namespace процесса

№ 7.2.3.2 — Docker internals

- **Что нужно знать:** UnionFS (AUFS, OverlayFS), слои, copy-on-write
- **Вопросы на собеседовании:** "Как Docker хранит данные на диске?"
- **Практика:** Проанализировать слои образа через docker history

№ 7.2.3.3 — Docker Registry

- **Что нужно знать:** приватный реестр, Docker Hub, Harbor

- **Вопросы на собеседовании:** "Как создать приватный Docker Registry?"
- **Практика:** Поднять приватный реестр и запустить образ

№ 7.2.3.4 — Docker Swarm

- **Что нужно знать:** оркестрация контейнеров (альтернатива K8s)
- **Вопросы на собеседовании:** "Чем Docker Swarm отличается от Kubernetes?"
- **Практика:** Создать Swarm-кластер из 3 узлов

№ 7.2.3.5 — Безопасность Docker

- **Что нужно знать:** сканирование уязвимостей (docker scan), подпись образов
- **Вопросы на собеседовании:** "Как проверить образ на уязвимости?"
- **Практика:** Запустить docker scan для образа и проанализировать результаты

№ 7.2.3.6 — Docker + Systemd

- **Что нужно знать:** запуск Docker-контейнеров как systemd-сервисы
- **Вопросы на собеседовании:** "Как автоматически запускать контейнеры при старте системы?"
- **Практика:** Создать systemd-сервис для контейнера

7.2.4. Практические задания (Junior → Senior Level)

№ 7.2.4.1

- **Задание:** Контейнеризовать C++ приложение с multi-stage сборкой
- **Что проверяет:** Знание Dockerfile

№ 7.2.4.2

- **Задание:** Создать docker-compose для приложения + PostgreSQL + Redis + Nginx
- **Что проверяет:** Знание Docker Compose

№ 7.2.4.3

- **Задание:** Оптимизировать Dockerfile для быстрой сборки (кэширование)

- **Что проверяет:** Знание кэширования

№ 7.2.4.4

- **Задание:** Настроить Healthcheck для API в Docker
 - **Что проверяет:** Знание healthcheck
-

РАЗДЕЛ 7.3: KUBERNETES (MIDDLE → STAFF)

7.3.1. Kubernetes (Middle Level)

№ 7.3.1.1 — Архитектура

- **Что нужно знать:** Master (API Server, etcd, Scheduler, Controller Manager), Nodes (kubelet, kube-proxy, container runtime)
- **Вопросы на собеседовании:** "Из каких компонентов состоит Kubernetes?"
- **Практика:** Нарисовать архитектуру K8s и объяснить каждый компонент

№ 7.3.1.2 — Pod

- **Что нужно знать:** минимальный деплой, один или несколько контейнеров, общий network namespace
- **Вопросы на собеседовании:** "Что такое Pod в Kubernetes?"
- **Практика:** Создать Pod с nginx и посмотреть его статус

№ 7.3.1.3 — Deployment

- **Что нужно знать:** управление версиями, rolling update, откат (rollback)
- **Вопросы на собеседовании:** "Как обновить приложение в K8s без простоя?"
- **Практика:** Создать Deployment с 3 репликами и обновить образ

№ 7.3.1.4 — Service

- **Что нужно знать:** ClusterIP (внутренний), NodePort (на всех нодах), LoadBalancer (внешний)
- **Вопросы на собеседовании:** "В чем разница между ClusterIP, NodePort и LoadBalancer?"
- **Практика:** Создать Service для доступа к приложению

№ 7.3.1.5 — ConfigMap

- **Что нужно знать:** хранение конфигурации (не секретной)
- **Вопросы на собеседовании:** "Как передать переменные окружения в Pod?"
- **Практика:** Создать ConfigMap и примонтировать как переменные

№ 7.3.1.6 — Secret

- **Что нужно знать:** хранение секретов (пароли, токены) в base64
- **Вопросы на собеседовании:** "Как хранить пароль в K8s?"
- **Практика:** Создать Secret для пароля БД

№ 7.3.1.7 — Ingress

- **Что нужно знать:** маршрутизация внешнего трафика, Ingress Controller (Nginx)
- **Вопросы на собеседовании:** "Как настроить доступ к приложению из интернета?"
- **Практика:** Создать Ingress для своего микросервиса

№ 7.3.1.8 — kubectl

- **Что нужно знать:** основные команды (get, describe, logs, exec, apply, delete)
- **Вопросы на собеседовании:** "Какие команды kubectl ты используешь?"
- **Практика:** Выполнить kubectl get pods и показать статусы

7.3.2. Kubernetes (Senior Level)

№ 7.3.2.1 — StatefulSet

- **Что нужно знать:** для stateful-приложений (БД), с PVC (Persistent Volume Claim)
- **Вопросы на собеседовании:** "Как развернуть базу данных в K8s?"
- **Практика:** Создать StatefulSet для PostgreSQL с PVC

№ 7.3.2.2 — DaemonSet

- **Что нужно знать:** запуск подов на каждой ноде (мониторинг)
- **Вопросы на собеседовании:** "Как запустить мониторинг на каждой ноде?"

- **Практика:** Создать DaemonSet для Prometheus node exporter

№ 7.3.2.3 — Persistent Volumes (PV) и Persistent Volume Claims (PVC)

- **Что нужно знать:** хранение данных
- **Вопросы на собеседовании:** "Как сохранить данные в K8s?"
- **Практика:** Создать PV и PVC для PostgreSQL

№ 7.3.2.4 — Helm

- **Что нужно знать:** пакетный менеджер, создание чартов, шаблонизация
- **Вопросы на собеседовании:** "Что такое Helm?"
- **Практика:** Создать Helm-чарт для своего микросервиса

№ 7.3.2.5 — Helm (chart)

- **Что нужно знать:** структура (Chart.yaml, values.yaml, templates/), helm install, helm upgrade
- **Вопросы на собеседовании:** "Как обновить приложение через Helm?"
- **Практика:** Установить и обновить чарт

№ 7.3.2.6 — Network Policies

- **Что нужно знать:** ограничение трафика между подами
- **Вопросы на собеседовании:** "Как ограничить доступ к БД только из приложения?"
- **Практика:** Создать Network Policy, разрешающую только приложению доступ к БД

№ 7.3.2.7 — HPA (Horizontal Pod Autoscaler)

- **Что нужно знать:** автоматическое масштабирование по CPU/метрикам
- **Вопросы на собеседовании:** "Как настроить автокасцирование?"
- **Практика:** Настроить HPA для микросервиса по CPU

№ 7.3.2.8 — Service Mesh

- **Что нужно знать:** Istio, Linkerd (управление трафиком, безопасность, наблюдаемость)
- **Вопросы на собеседовании:** "Что такое Service Mesh?"
- **Практика:** Установить Istio и настроить маршрутизацию

№ 7.3.2.9 — Operators

- **Что нужно знать:** автоматизация управления приложениями (PostgreSQL Operator, Prometheus Operator)
- **Вопросы на собеседовании:** "Что такое Operator в K8s?"
- **Практика:** Установить PostgreSQL Operator и создать кластер

№ 7.3.2.10 — Istio (VirtualService, DestinationRule)

- **Что нужно знать:** управление трафиком, canary deployments
 - **Вопросы на собеседовании:** "Как реализовать canary deployment?"
 - **Практика:** Настроить VirtualService для 90/10 трафика
-

7.3.3. Kubernetes (Staff Level)

№ 7.3.3.1 — Внутреннее устройство etcd

- **Что нужно знать:** хранение состояния кластера
- **Вопросы на собеседовании:** "Как etcd используется в K8s?"
- **Практика:** Изучить etcd и его роль в K8s

№ 7.3.3.2 — Scheduler

- **Что нужно знать:** как работает, кастомный scheduler
- **Вопросы на собеседовании:** "Как K8s выбирает ноду для Pod?"
- **Практика:** Изучить алгоритм работы scheduler

№ 7.3.3.3 — Controller Manager

- **Что нужно знать:** контроллеры (Deployment, ReplicaSet, StatefulSet)
- **Вопросы на собеседовании:** "Как работает Deployment Controller?"
- **Практика:** Изучить внутреннее устройство Deployment

№ 7.3.3.4 — CRD (Custom Resource Definitions)

- **Что нужно знать:** создание своих ресурсов
- **Вопросы на собеседовании:** "Как создать свой ресурс в K8s?"
- **Практика:** Создать CRD MyApp и написать controller

№ 7.3.3.5 — API Server

- **Что нужно знать:** аутентификация, авторизация (RBAC), admission controllers

- **Вопросы на собеседовании:** "Как работает авторизация в K8s?"
- **Практика:** Настроить RBAC для ограничения доступа

№ 7.3.3.6 — Kubelet

- **Что нужно знать:** управление подами, взаимодействие с runtime (containerd, CRI-O)
- **Вопросы на собеседовании:** "Как Kubelet взаимодействует с container runtime?"
- **Практика:** Изучить архитектуру Kubelet

№ 7.3.3.7 — Pod Lifecycle

- **Что нужно знать:** Init Containers, PostStart, PreStop, readiness/startup/liveness probes
- **Вопросы на собеседовании:** "Как настроить graceful shutdown в K8s?"
- **Практика:** Добавить PreStop hook и probe

№ 7.3.3.8 — Node affinity и Taints/Tolerations

- **Что нужно знать:** управление размещением подов
- **Вопросы на собеседовании:** "Как зарезервировать ноду для определенного приложения?"
- **Практика:** Настроить taints и tolerations

№ 7.3.3.9 — Cluster autoscaling

- **Что нужно знать:** автоматическое добавление/удаление нод
- **Вопросы на собеседовании:** "Как работает Cluster Autoscaler?"
- **Практика:** Настроить Cluster Autoscaler в облаке

7.3.4. Практические задания (Middle → Staff Level)

№ 7.3.4.1

- **Задание:** Развернуть микросервис в minikube с 3 репликами и настроить Ingress
- **Что проверяет:** Знание базового K8s

№ 7.3.4.2

- **Задание:** Создать Helm-чарт для микросервиса с configurable параметрами
- **Что проверяет:** Знание Helm

№ 7.3.4.3

- **Задание:** Настроить HPA по CPU и протестировать автокасцирование
- **Что проверяет:** Знание масштабирования

№ 7.3.4.4

- **Задание:** Настроить Istio для canary deployment (90/10)
- **Что проверяет:** Знание Service Mesh

№ 7.3.4.5

- **Задание:** Создать PostgreSQL StatefulSet с PVC и настроить резервное копирование
- **Что проверяет:** Знание StatefulSet

РАЗДЕЛ 7.4: CI/CD (JUNIOR → SENIOR)

7.4.1. CI/CD (Middle Level)

№ 7.4.1.1 — CI/CD

- **Что нужно знать:** Continuous Integration (сборка, тесты) vs Continuous Delivery/Deployment
- **Вопросы на собеседовании:** "В чем разница между CI и CD?"
- **Практика:** Объяснить концепцию CI/CD

№ 7.4.1.2 — GitLab CI

- **Что нужно знать:** .gitlab-ci.yml, stages, jobs, artifacts, cache
- **Вопросы на собеседовании:** "Как настроить пайплайн в GitLab?"
- **Практика:** Написать пайплайн для сборки C++ проекта

№ 7.4.1.3 — GitHub Actions

- **Что нужно знать:** on, jobs, steps, runs-on, actions/checkout
- **Вопросы на собеседовании:** "Как настроить пайплайн в GitHub Actions?"
- **Практика:** Написать пайплайн для сборки C# проекта

№ 7.4.1.4 — Артефакты

- **Что нужно знать:** сохранение результатов сборки (бинарники, отчеты о тестах)
- **Вопросы на собеседовании:** "Как сохранить бинарники после сборки?"
- **Практика:** Сохранить артефакты в GitLab CI

№ 7.4.1.5 — Кеширование

- **Что нужно знать:** ускорение сборки (кэш зависимостей)
- **Вопросы на собеседовании:** "Как ускорить сборку в CI?"
- **Практика:** Настроить кеширование для NuGet/Conan

№ 7.4.1.6 — Docker в CI

- **Что нужно знать:** сборка образа и пуш в registry
 - **Вопросы на собеседовании:** "Как собирать и пушить Docker-образы в CI?"
 - **Практика:** Добавить сборку Docker-образа в пайплайн
-

7.4.2. CI/CD (Senior Level)

№ 7.4.2.1 — GitOps

- **Что нужно знать:** декларативный подход, ArgoCD, синхронизация состояния
- **Вопросы на собеседовании:** "Что такое GitOps?"
- **Практика:** Настроить ArgoCD для деплоя из Git

№ 7.4.2.2 — Артефакты (Docker Registry)

- **Что нужно знать:** хранение образов, версионирование, очистка
- **Вопросы на собеседовании:** "Как управлять версиями Docker-образов?"
- **Практика:** Настроить тегирование образов (git commit sha)

№ 7.4.2.3 — Multi-environment

- **Что нужно знать:** dev, stage, prod с разными конфигурациями
- **Вопросы на собеседовании:** "Как деплоить в несколько окружений?"

- **Практика:** Настроить деплой в dev/stage/prod в GitLab CI

№ 7.4.2.4 — Rollback

- **Что нужно знать:** автоматический или ручной откат при неудаче
- **Вопросы на собеседовании:** "Как реализовать откат в CI/CD?"
- **Практика:** Настроить автоматический rollback в ArgoCD

№ 7.4.2.5 — Security

- **Что нужно знать:** сканирование уязвимостей (Trivy, Snyk) в CI
- **Вопросы на собеседовании:** "Как проверять образы на уязвимости?"
- **Практика:** Добавить Trivy сканирование в пайплайн

№ 7.4.2.6 — Notifications

- **Что нужно знать:** отправка уведомлений (Slack, Telegram) о статусе сборки
- **Вопросы на собеседовании:** "Как отправлять уведомления о сборке?"
- **Практика:** Добавить отправку в Slack в GitLab CI

7.4.3. Практические задания (Middle → Senior Level)

№ 7.4.3.1

- **Задание:** Настроить GitLab CI для сборки C++ проекта с CMake и запуском тестов
- **Что проверяет:** Знание CI

№ 7.4.3.2

- **Задание:** Настроить GitHub Actions для сборки, тестирования и публикации Docker-образа
- **Что проверяет:** Знание CI/CD

№ 7.4.3.3

- **Задание:** Настроить ArgoCD для деплоя микросервиса из Git
- **Что проверяет:** Знание GitOps

№ 7.4.3.4

- **Задание:** Добавить Trivy сканирование уязвимостей в CI
- **Что проверяет:** Знание безопасности

РАЗДЕЛ 7.5: OBSERVABILITY (SENIOR → STAFF)

7.5.1. Метрики, Логи, Трейсы (Senior Level)

№ 7.5.1.1 — Observability

- **Что нужно знать:** три столпа (метрики, логи, трейсы)
- **Вопросы на собеседовании:** "Что такое Observability?"
- **Практика:** Объяснить три столпа

№ 7.5.1.2 — Prometheus

- **Что нужно знать:** сбор метрик, экспортеры (node_exporter, cadvisor), prometheus.yml
- **Вопросы на собеседовании:** "Как собирать метрики в Prometheus?"
- **Практика:** Поднять Prometheus и настроить сбор метрик

№ 7.5.1.3 — Grafana

- **Что нужно знать:** дашборды, визуализация, AlertManager
- **Вопросы на собеседовании:** "Как создавать дашборды в Grafana?"
- **Практика:** Создать дашборд для CPU, памяти, ошибок

№ 7.5.1.4 — AlertManager

- **Что нужно знать:** настройка алертов, отправка в Slack/Email
- **Вопросы на собеседовании:** "Как настроить оповещения?"
- **Практика:** Настроить алерт на высокий CPU

№ 7.5.1.5 — OpenTelemetry

- **Что нужно знать:** стандарт для трейсинга, метрик, логов
- **Вопросы на собеседовании:** "Что такое OpenTelemetry?"
- **Практика:** Добавить OpenTelemetry в [ASP.NET](#) Core

№ 7.5.1.6 — Jaeger/Tempo

- **Что нужно знать:** распределенный трейсинг
- **Вопросы на собеседовании:** "Как увидеть полный путь запроса?"
- **Практика:** Развернуть Jaeger и увидеть трейсы

№ 7.5.1.7 — ELK / OpenSearch

- **Что нужно знать:** централизованное логирование (Elasticsearch, Logstash, Kibana)
- **Вопросы на собеседовании:** "Как организовать централизованное логирование?"
- **Практика:** Настроить ELK для микросервисов

№ 7.5.1.8 — Fluent-bit / Logstash

- **Что нужно знать:** сбор логов и отправка в Elasticsearch
 - **Вопросы на собеседовании:** "Как собирать логи с серверов?"
 - **Практика:** Настроить Fluent-bit для сбора логов приложений
-

7.5.2. SRE и мониторинг (Staff Level)

№ 7.5.2.1 — SLA / SLO / SLI

- **Что нужно знать:** Service Level Agreement, Objective, Indicator
- **Вопросы на собеседовании:** "В чем разница между SLA, SLO и SLI?"
- **Практика:** Написать SLO для сервиса (доступность 99.95%)

№ 7.5.2.2 — Error Budget

- **Что нужно знать:** допустимое количество ошибок в рамках SLO
- **Вопросы на собеседовании:** "Что такое Error Budget?"
- **Практика:** Рассчитать Error Budget для 99.95% доступности

№ 7.5.2.3 — Blame-less Post Mortem

- **Что нужно знать:** анализ инцидентов без поиска виноватых
- **Вопросы на собеседовании:** "Как анализировать инциденты?"
- **Практика:** Написать постмортем на инцидент

№ 7.5.2.4 — On-call

- **Что нужно знать:** организация дежурств, escalation policies
- **Вопросы на собеседовании:** "Как организовать дежурства?"
- **Практика:** Настроить PagerDuty для on-call

№ 7.5.2.5 — Chaos Engineering

- **Что нужно знать:** тестирование устойчивости (Chaos Monkey, Gremlin)

- **Вопросы на собеседовании:** "Что такое Chaos Engineering?"
- **Практика:** Провести chaos-тест для проверки отказоустойчивости

№ 7.5.2.6 — Capacity Planning

- **Что нужно знать:** планирование мощности, масштабирование
- **Вопросы на собеседовании:** "Как планировать мощность для будущего роста?"
- **Практика:** Проанализировать тренды использования ресурсов

7.5.3. Практические задания (Senior → Staff Level)

№ 7.5.3.1

- **Задание:** Поднять Prometheus + Grafana и создать дашборд для микросервиса
- **Что проверяет:** Знание мониторинга

№ 7.5.3.2

- **Задание:** Настроить OpenTelemetry в [ASP.NET](#) Core и отправить трейсы в Jaeger
- **Что проверяет:** Знание Observability

№ 7.5.3.3

- **Задание:** Настроить ELK для сбора логов и дашборд в Kibana
- **Что проверяет:** Знание логирования

№ 7.5.3.4

- **Задание:** Написать постмортем на реальный или вымышленный инцидент
- **Что проверяет:** Знание SRE

☞ СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

Linux: навигация, права, процессы, переменные окружения

Bash: переменные, условия, циклы, grep, awk, sed

Docker: образы, контейнеры, Dockerfile, docker run, docker ps, docker exec

Docker Compose: многоконтейнерные приложения

Middle Level (должен знать 100%)

Docker: multi-stage, .dockerignore, healthcheck, оптимизация кэширования

Kubernetes: Pod, Deployment, Service (ClusterIP/NodePort/LoadBalancer)

Kubernetes: ConfigMap, Secret, Ingress

kubectl: основные команды

CI/CD: GitLab CI / GitHub Actions (базовые пайплайны)

Prometheus + Grafana: базовый мониторинг

Senior Level (должен знать 100%)

Kubernetes: StatefulSet, DaemonSet, PV/PVC, Helm, HPA

Kubernetes: Network Policies, Service Mesh (Istio)

CI/CD: GitOps (ArgoCD), multi-environment, rollback

Observability: OpenTelemetry, Jaeger, ELK

Docker: internals (cgroups, namespaces, UnionFS)

Staff Level (должен знать 100%)

Kubernetes: внутреннее устройство (etcd, scheduler, controller manager)

Kubernetes: CRD, Operators, API Server internals

CI/CD: безопасность, сканирование уязвимостей

SRE: SLA/SLO/SLI, Error Budget, Blame-less Post Mortem

Chaos Engineering

Capacity Planning

🔨 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 7

Задание: Разработать CI/CD пайплайн и инфраструктуру для микросервисного приложения.

Шаги:

1. **Приложение:** микросервис на C# ([ASP.NET](#) Core) + PostgreSQL
2. **Docker:** multi-stage Dockerfile для приложения
3. **Kubernetes:** StatefulSet для PostgreSQL, Deployment для приложения, Service, Ingress
4. **Helm:** создать чарт для приложения
5. **CI/CD:** GitLab CI пайплайн: сборка, тесты, сборка образа, пуш в registry, деплой в dev/stage
6. **GitOps:** ArgoCD для деплоя из Git
7. **Observability:** Prometheus + Grafana (метрики), Jaeger (трейсы), ELK (логи)
8. **Мониторинг:** алерты на высокий CPU, ошибки
9. **Тестирование:** написать SLO для сервиса (доступность 99.9%)

Проверка:

- Пайплайн должен проходить за < 5 минут
- Приложение должно автоматически масштабироваться по CPU
- При падении контейнера, он должен автоматически перезапускаться
- Все метрики, трейсы и логи должны собираться в централизованные системы